

Actividad 08

Laboratorio: File Path Traversal, Traversal Sequences Blocked with Absolute Path Bypass

PortSwigger Web Security Academy

Elaborado por: Juan Pablo Casanova De La Madrid

Herramienta utilizada: Burp Suite Community Edition

1. Nombre del laboratorio

File path traversal, traversal sequences blocked with absolute path bypass

Laboratorio perteneciente a PortSwigger Web Security Academy, dentro de la categoría File path traversal.

2. Nivel

Practitioner.

A diferencia de los laboratorios de nivel Apprentice, en este caso la aplicación incorpora una mitigación básica: filtra y elimina las secuencias de retroceso de directorio ("../") que el usuario intente enviar en el parámetro vulnerable. Por ello, la explotación directa mediante ../../../../etc/passwd no funciona, y es necesario identificar una técnica de bypass adicional (en este caso, el uso de una ruta absoluta) para completar el laboratorio.

3. Tipo de explotación

Path Traversal / Directory Traversal con bypass de filtro (CWE-22: Improper Limitation of a Pathname to a Restricted Directory).

Es una vulnerabilidad del lado del servidor que permite a un atacante manipular la ruta de un archivo solicitado para acceder a archivos fuera del directorio raíz previsto por la aplicación. En este laboratorio, la explotación requiere además evadir el filtro anti-traversal implementado por la aplicación, aprovechando que dicho filtro solo contempla secuencias relativas ("../") y no valida el caso en que se envía directamente una ruta absoluta del sistema de archivos.

4. Objetivo del laboratorio

El laboratorio contiene una vulnerabilidad de path traversal en la funcionalidad que muestra las imágenes de los productos de la tienda. La aplicación bloquea las secuencias de traversal, pero trata el nombre de archivo recibido como relativo a un directorio de trabajo por defecto.

Objetivo: explotar dicha vulnerabilidad, evadiendo el filtro de secuencias de traversal mediante una ruta absoluta, para recuperar el contenido del archivo del sistema /etc/passwd, lo que confirma la lectura arbitraria de archivos y da por resuelto el laboratorio.



Figura 1. Descripción y objetivo del laboratorio en PortSwigger Web Security Academy (nivel Practitioner, estado Not solved).

5. Explicación técnica de la vulnerabilidad

La aplicación web muestra el catálogo de productos ("We Like To Shop") y cada imagen se sirve dinámicamente mediante un endpoint del tipo:

GET /image?filename=<nombre_de_archivo>

El parámetro filename es utilizado por el servidor para construir, de forma interna, la ruta física del archivo a devolver, concatenándolo con un directorio base de imágenes por defecto. A diferencia de un caso simple, aquí la aplicación sí intenta mitigar el path traversal: antes de usar el valor recibido, elimina o filtra las secuencias de retroceso de directorio (".." o "..\") que el usuario incluya en filename.

Sin embargo, este control solo contempla rutas relativas. El filtro no valida ni bloquea el caso en que el valor recibido sea directamente una ruta absoluta del sistema de archivos (por ejemplo, /etc/passwd en sistemas Linux). Como consecuencia, cuando la aplicación recibe una ruta absoluta, la trata directamente como la ubicación del archivo a leer, ignorando el directorio base de imágenes al que debería estar restringida.

Esta vulnerabilidad se clasifica dentro de CWE-22 y es un ejemplo representativo de cómo un filtro incompleto (que cubre únicamente el patrón de ataque más común) puede ser evadido mediante una variante que el desarrollador no contempló, permitiendo la lectura arbitraria de archivos del servidor.

6. Procedimiento paso a paso

1. Se inicia Burp Suite Community Edition, seleccionando "Temporary project in memory" y la configuración por defecto ("Use Burp defaults").

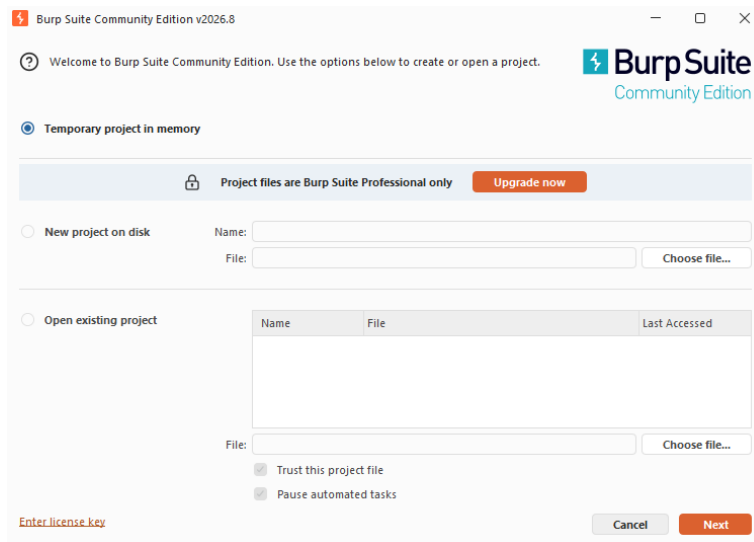


Figura 2. Creación de un proyecto temporal en memoria.

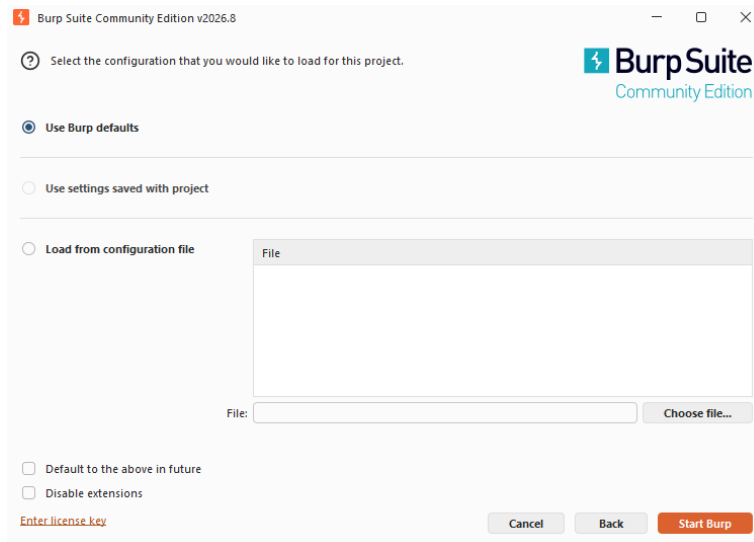


Figura 3. Selección de la configuración por defecto de Burp.

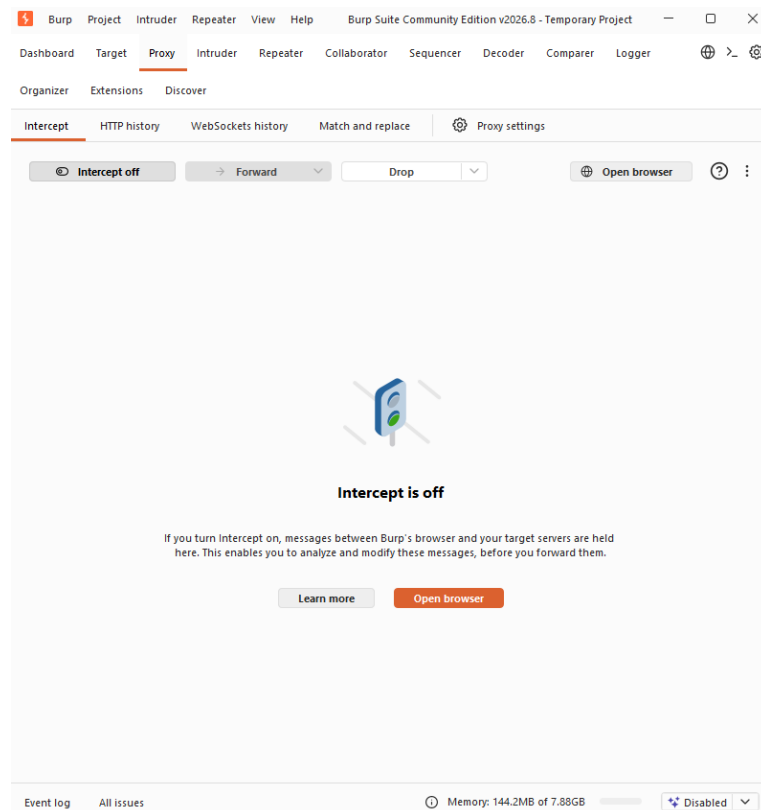


Figura 4. Burp Suite iniciado, pestaña Proxy con Intercept inicialmente apagado.

2. Se abre el navegador integrado de Burp y se accede a la URL del laboratorio proporcionada por PortSwigger.

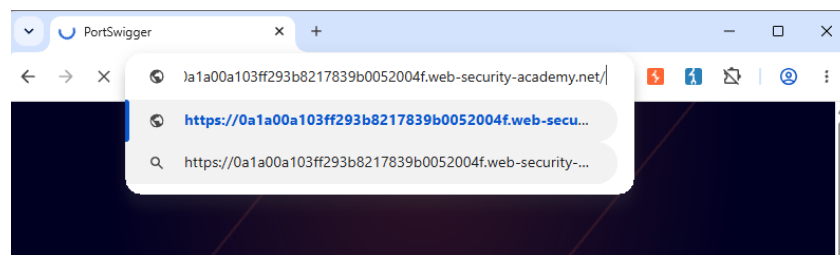


Figura 5. Navegación hacia la URL del laboratorio a través del navegador de Burp.

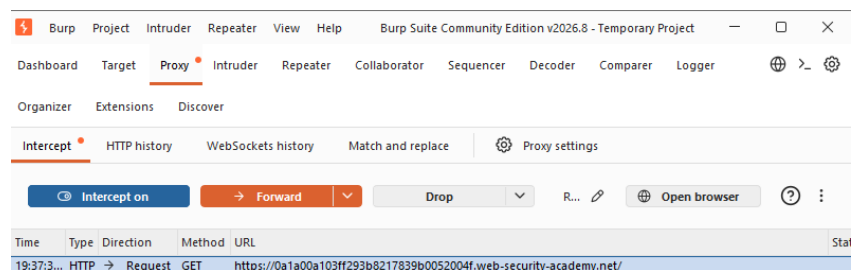


Figura 6. Petición GET a la raíz del sitio capturada en el historial del Proxy.

3. Con el Intercept activo, se navega por la tienda; Burp captura las peticiones GET que la página realiza para cargar cada imagen de producto, del tipo `/image?filename=NN.jpg`, confirmando que el nombre de archivo se maneja mediante un parámetro controlable por el cliente.

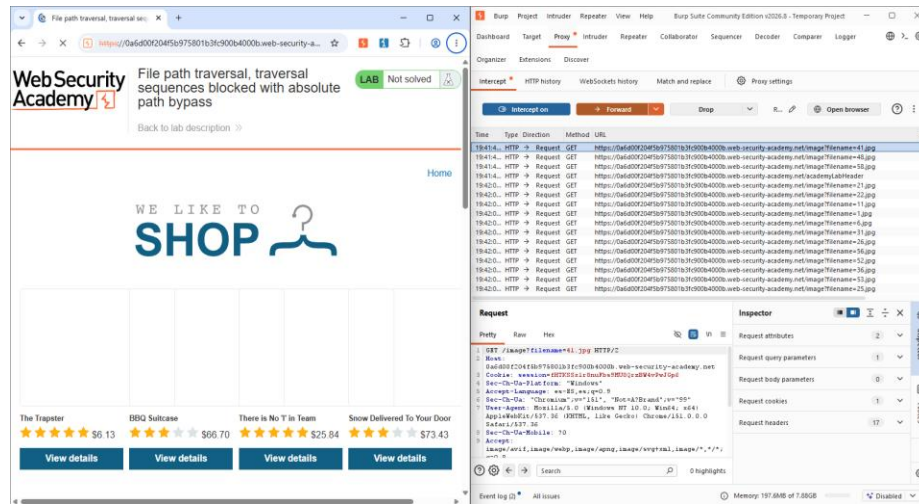


Figura 7. Petición interceptada `GET /image?filename=41.jpg` junto con el resto de peticiones de imágenes en el historial del Proxy.

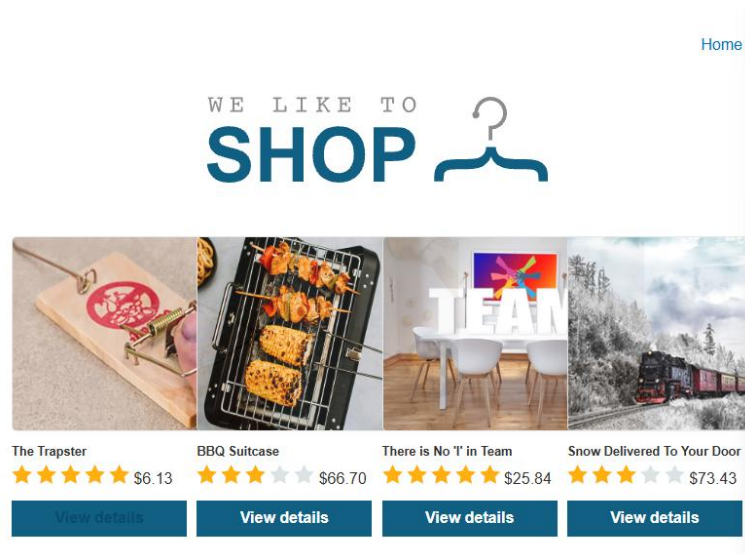


Figura 8. Vista de la tienda mostrando los productos cuyas imágenes se cargan mediante el parámetro `filename`.

4. Se identifica una petición hacia una imagen de producto (filename=5.jpg) y, mediante clic derecho, se envía al módulo Repeater ("Send to Repeater") para poder modificarla y reenviarla libremente.

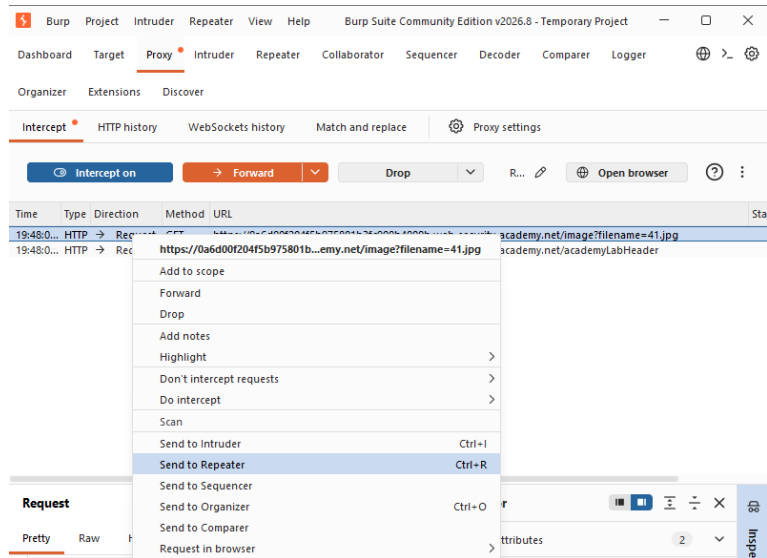


Figura 9. Menú contextual: envío de la petición interceptada a Repeater.

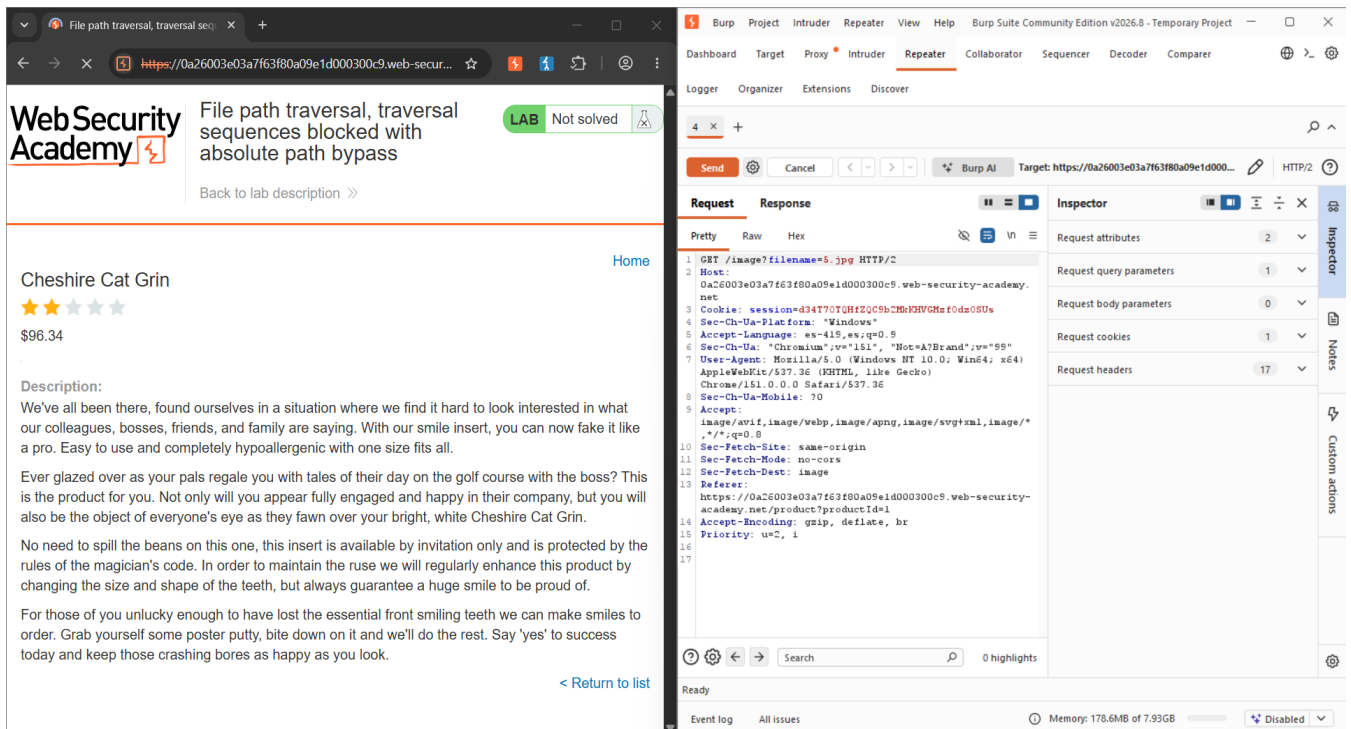


Figura 10. Petición original GET /image?filename=5.jpg ya cargada en la pestaña Repeater.

- En Repeater se reemplaza el valor 5.jpg por la ruta absoluta /etc/passwd, con el fin de evadir el filtro de secuencias relativas y forzar al servidor a acceder directamente al archivo de credenciales del sistema.
GET /image?filename=/etc/passwd HTTP/2

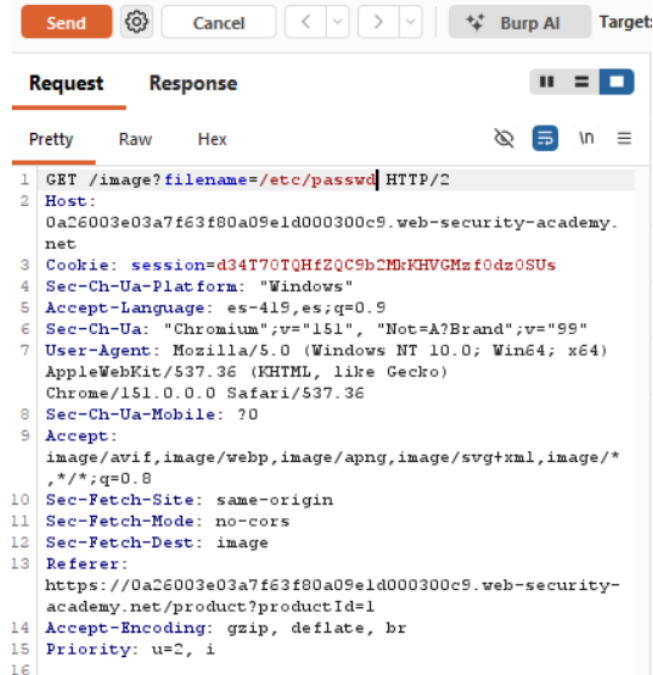


Figura 11. Payload de path traversal con ruta absoluta inyectado en el parámetro filename.

- Se hace clic en Send y se analiza la respuesta del servidor.
- El servidor responde con código HTTP/2 200 OK y devuelve en el cuerpo de la respuesta el contenido íntegro del archivo /etc/passwd, confirmando la explotación exitosa de la vulnerabilidad.

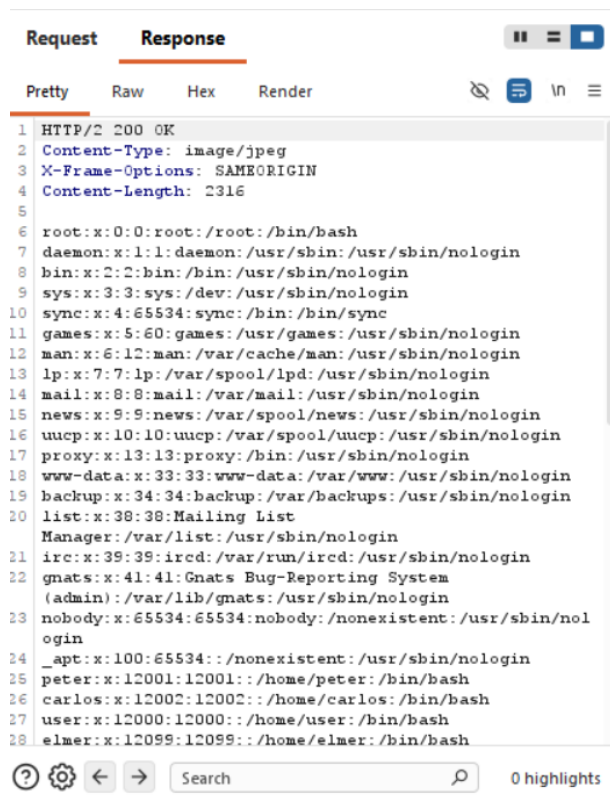


Figura 12. Respuesta vulnerable del servidor (200 OK) mostrando el contenido de /etc/passwd.

8. Finalmente, se regresa a la página del laboratorio en PortSwigger, la cual actualiza automáticamente su estado a "Solved", confirmando que el objetivo de la actividad fue completado.



Figura 13. Confirmación de laboratorio resuelto ("Solved").

7. Explicación del payload

GET /image?filename=/etc/passwd HTTP/2

El payload se construye a partir de un único componente clave:

- **/etc/passwd:** en lugar de enviar un nombre de archivo relativo (5.jpg) o una secuencia de traversal relativa (../../../../etc/passwd, filtrada por la aplicación), se envía directamente una ruta absoluta de Linux. Al recibir una ruta que comienza con "/", el sistema operativo la interpreta desde la raíz del sistema de archivos, sin importar el directorio de trabajo por defecto que la aplicación tenía pensado usar como base.
- **Evasión del filtro:** el mecanismo de protección de la aplicación solo busca y elimina patrones como "../", por lo que una ruta absoluta simplemente no contiene esos patrones y pasa el control sin ser modificada ni bloqueada.

Al concatenar (o directamente usar) el payload en lugar de la ruta base que la aplicación esperaba, el resultado final resuelto por el sistema operativo equivale a /etc/passwd, fuera del directorio que la aplicación pretendía exponer. /etc/passwd es un archivo clásico usado como prueba de concepto en pentesting de Linux, ya que es legible por cualquier usuario del sistema y su contenido confirma de forma clara e inequívoca la lectura arbitraria de archivos.

8. Mitigación recomendada

Para prevenir vulnerabilidades de path traversal como la explotada en este laboratorio, se recomienda aplicar las siguientes medidas, de forma combinada:

- Validar el parámetro filename contra una lista blanca (whitelist) de nombres de archivo o identificadores permitidos, en lugar de intentar detectar y bloquear patrones maliciosos específicos.
- Rechazar cualquier valor de entrada que contenga separadores de ruta ("/" o "\") o que comience con "/" (ruta absoluta) o con una letra de unidad (en Windows), en vez de limitarse a eliminar únicamente las secuencias "../".
- Resolver la ruta final de forma canónica (canonicalización) con funciones seguras del lenguaje o framework, y verificar después de resolverla que el resultado siga estando dentro del directorio base permitido (comparación de prefijo sobre la ruta absoluta ya normalizada).
- Evitar exponer directamente el sistema de archivos del servidor a partir de datos proporcionados por el usuario; en su lugar, mapear identificadores de producto/imagen a rutas físicas mediante una tabla o índice controlado por el backend.
- Ejecutar el proceso de la aplicación con el mínimo privilegio necesario, restringiendo los permisos de lectura del usuario del servicio web únicamente a los directorios estrictamente necesarios.
- Reforzar la defensa en profundidad con un WAF (Web Application Firewall) que detecte y bloquee patrones de path traversal (incluyendo rutas absolutas y variantes codificadas), y aplicar monitoreo/alertas sobre accesos a archivos fuera de lo esperado.